МТУСИ

Московский технический университет
связи и информатики

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Разработка REST API»

Дисциплина: «Программная инженерия»

Вариант 5. API фильмотеки

Выполнил: студент группы БСТ-2556
Новосельцев Алексей Сергеевич

Проверил: _____

Москва, 2026

1. Цель работы и индивидуальное задание

Цель работы - освоить разработку REST API на Python, применение HTTP-методов GET, POST, PUT и DELETE, маршрутизацию, сериализацию JSON и хранение данных в реляционной базе.

Индивидуальное задание: вариант 5, API фильмотеки. Требуется вести учёт фильмов, выполнять поиск по названию и фильтрацию по году и рейтингу. Основная сущность проекта - фильм.

Требование	Реализация в проекте
Python, веб-фреймворк	Python 3.12, FastAPI
Хранение в базе данных	SQLite, ORM SQLAlchemy 2
CRUD основной сущности	POST, GET, PUT, DELETE для фильмов
Не менее 7 эндпоинтов	8 операций API, без учёта документации
Поиск, фильтрация или агрегация	Поиск по названию; год, рейтинг, жанр; статистика
Проверка ввода	Pydantic, ограничения CHECK в SQLite
Сортировка, пагинация или авторизация	Сортировка по 4 полям и постраничная выдача
Документация	Swagger UI, ReDoc и OpenAPI JSON

2. Краткие теоретические сведения

REST описывает взаимодействие клиента и сервера через ресурсы с адресами URI. В данном проекте коллекция фильмов доступна по /api/v1/movies, отдельный фильм - по этому же пути с идентификатором. Каждый запрос содержит сведения, необходимые для его обработки.

GET читает ресурс, POST создаёт, PUT полностью заменяет редактируемые поля существующего ресурса, DELETE удаляет. Состояние обработки передаётся HTTP-кодом, данные - в JSON. FastAPI формирует OpenAPI-схему на основе объявленных маршрутов и моделей [1].

3. Архитектура и структура проекта

Клиент обращается к FastAPI по HTTP. Pydantic проверяет параметры и тело запроса, обработчик выполняет запрос через SQLAlchemy, а SQLite сохраняет изменения. Ответ проверяется схемой и сериализуется в JSON.

```
HTTP-клиент / Swagger UI
          | запрос JSON
          v
FastAPI -> Pydantic -> SQLAlchemy Session -> SQLite
          | ответ JSON и HTTP-код
          v
        Клиент
```

Файл / каталог	Назначение
app/main.py	Создание FastAPI, жизненный цикл, подключение маршрутов
app/database.py	Движок SQLite, Base, сессия БД для запроса
app/models.py	ORM-модель Movie, индексы, CHECK-ограничения
app/schemas.py	Модели ввода и ответа, фильтры и валидация
app/routes.py	8 операций API и формирование SQL-запросов
app/seed.py	Заполнение пустой БД учебными данными
tests/test_api.py	Автоматические проверки на временных базах
examples/	Данные JSON и готовые HTTP-запросы
docs/	OpenAPI, ответы API, результаты pytest, снимок Swagger
scripts/	Запись демонстрационных ответов и сборка PDF

Таблица создаётся при запуске приложения. Сессия открывается для HTTP-запроса и закрывается после его завершения. Изменяющие операции фиксируются через commit(). Файл data/movies.db сохраняется между запусками, поэтому состояние не зависит от памяти процесса [1, 3].

В репозиторий включены восемь вымышленных учебных фильмов. Их названия, режиссёры и рейтинги служат для демонстрации и не являются внешними справочными данными.

4. Модель данных и валидация

Таблица movies содержит один фильм в каждой строке. Идентификатор формируется базой. Для года и рейтинга созданы индексы. Поля ORM описывают хранение, а схемы Pydantic - допустимый ввод и формат JSON [3, 4].

Поле	Тип в SQLite	Ограничение / назначение
id	INTEGER	Первичный ключ
title	VARCHAR(200)	Название, 1-200 символов
director	VARCHAR(120)	Режиссёр, 1-120 символов
year	INTEGER	Год выпуска, 1888-2100
genre	VARCHAR(60)	Жанр, 1-60 символов
rating	FLOAT	Учебный рейтинг, 0-10 включительно
duration_minutes	INTEGER	Длительность, 1-1000 минут
description	TEXT, NULL	Описание до 3000 символов, необязательное

Пробелы по краям строк удаляются до проверки длины. Жанры приводятся к единой записи: « дРАМА » преобразуется в «Драма». Пустые обязательные строки, неизвестные поля, неверные типы и значения вне диапазонов отклоняются с кодом 422.

```
class MovieWrite(BaseModel):
    model_config = ConfigDict(
        str_strip_whitespace=True,
        extra="forbid", allow_inf_nan=False
    )
    title: str = Field(min_length=1, max_length=200)
    year: int = Field(ge=1888, le=2100, strict=True)
    rating: float = Field(ge=0, le=10, strict=True)
    duration_minutes: int = Field(ge=1, le=1000, strict=True)
```

Фрагмент показывает основные ограничения; полный набор полей находится в app/schemas.py. Числа JSON проверяются строго: строка «8.2» или true не заменяют числовой рейтинг. Минимальная граница рейтинга в фильтре не может превышать максимальную.

Ограничения диапазонов и длины строк продублированы через CHECK. Это позволяет базе отклонять ошибочные значения и при обходе HTTP-валидации.

5. Маршруты и HTTP-коды

Версия предметного API включена в URI: /api/v1. Ниже приведены восемь реализованных операций. Автоматические /docs, /redoc, /openapi.json и перенаправление с / в этот перечень не включены.

Метод	Путь	Результат	Код
GET	/health	Проверка API и БД	200
GET	/api/v1/movies	Список фильмов с параметрами	200
POST	/api/v1/movies	Созданный фильм с id	201
GET	/api/v1/movies/{movie_id}	Один фильм	200
PUT	/api/v1/movies/{movie_id}	Полностью обновлённый фильм	200
DELETE	/api/v1/movies/{movie_id}	Пустое тело ответа	204
GET	/api/v1/movies/stats	Статистика всей базы	200
GET	/api/v1/genres	Список используемых жанров	200

После POST сервер возвращает объект с id и заголовок Location, указывающий путь к созданному ресурсу. PUT принимает все обязательные редактируемые поля; если description пропущен, описание становится null. DELETE возвращает 204 без тела.

Если фильм отсутствует, GET, PUT и DELETE по его идентификатору возвращают 404 с сообщением «Фильм не найден». Некорректное тело, ID или параметры списка дают 422 с описанием поля ошибки.

```
{
  "detail": "Фильм не найден"
}
```

Статический путь /movies/stats объявлен раньше динамического /movies/{movie_id}; слово stats поэтому не воспринимается как числовой ID. /health выполняет SELECT 1, проверяя фактическое соединение с базой.

Статистика включает количество фильмов, средний рейтинг, минимальный и максимальный год, а также число фильмов и средний рейтинг по каждому жанру. Для пустой БД числовые агрегаты без значения возвращаются как null, количество равно нулю.

6. Поиск, фильтры, сортировка и пагинация

Параметр	Правило
q	Часть названия, 1-200 символов, без учёта регистра
year	Точный год в диапазоне 1888-2100
genre	Точное совпадение жанра без учёта регистра
rating_min / rating_max	Нижняя и верхняя границы рейтинга, 0-10 включительно
sort_by	id, title, year или rating; по умолчанию id
order	asc или desc; по умолчанию asc
page	Номер страницы от 1 до 1 000 000 000; по умолчанию 1
page_size	Размер страницы 1-100; по умолчанию 10

Условия поиска и фильтрации объединяются через AND. Для кириллицы в SQLite зарегистрирована функция casefold(). Поиск использует параметризованное выражение contains(..., autoescape=True), поэтому символы % и _ считаются буквальными.

Поле сортировки выбирается из белого списка. При равных значениях дополнительно применяется id по возрастанию. Смещение вычисляется как (page - 1) × page_size. Число страниц равно округлению total / page_size вверх.

```
GET /api/v1/movies?q=ОРБИТА&year=2024&rating_min=8
    &sort_by=rating&order=desc&page_size=2
```

Название в фактическом ответе	Год	Рейтинг
Орбита памяти	2024	8.8
Орбита надежды	2024	8.4

Метаданные ответа: total=2, page=1, page_size=2, total_pages=1. Два фильма найдены по заглавному запросу «ОРБИТА» и расположены по убыванию рейтинга.

При отсутствии совпадений возвращаются пустой items и total_pages=0. Для страницы за пределами выборки items пуст, но total отражает фактическое число подходящих записей.

7. Реализация CRUD-операций

Ниже показаны фрагменты app/routes.py. При создании Pydantic-модель преобразуется в словарь, из него строится ORM-объект. После фиксации транзакции refresh() получает значения, назначенные базой.

```
def create_movie(payload, response, session):
    movie = Movie(**payload.model_dump())
    session.add(movie)
    session.commit()
    session.refresh(movie)
    response.headers["Location"] = f"/api/v1/movies/{movie.id}"
    return movie
```

Чтение выполняется через session.get(Movie, movie_id). Если объект не найден, обработчик возбуждает HTTPException с кодом 404. Для ответа используется схема MovieRead с поддержкой ORM-атрибутов.

```
def replace_movie(movie_id, payload, session):
    movie = find_movie(session, movie_id)
    for field, value in payload.model_dump().items():
        setattr(movie, field, value)
    session.commit()
    session.refresh(movie)
    return movie

def delete_movie(movie_id, session):
    session.delete(find_movie(session, movie_id))
    session.commit()
    return Response(status_code=204)
```

Для краткости в этих листингах опущены декораторы и аннотации типов; полный исполняемый код находится в проекте. Декораторы @router.post, @router.get, @router.put и @router.delete связывают функции с HTTP-методами и путями.

Параметризованные SQL-запросы отделяют значения пользователя от структуры SQL. Выборка выполняется с LIMIT и OFFSET, поэтому API не выгружает всю коллекцию на каждую страницу.

8. Запуск приложения и документация

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m app.seed
python -m uvicorn app.main:app --reload
```

Требуется Python 3.12+. Команды выполняются из папки проекта. В Windows активация: .venv\Scripts\activate. Swagger UI: http://127.0.0.1:8000/docs; ReDoc: /redoc; схема: /openapi.json.

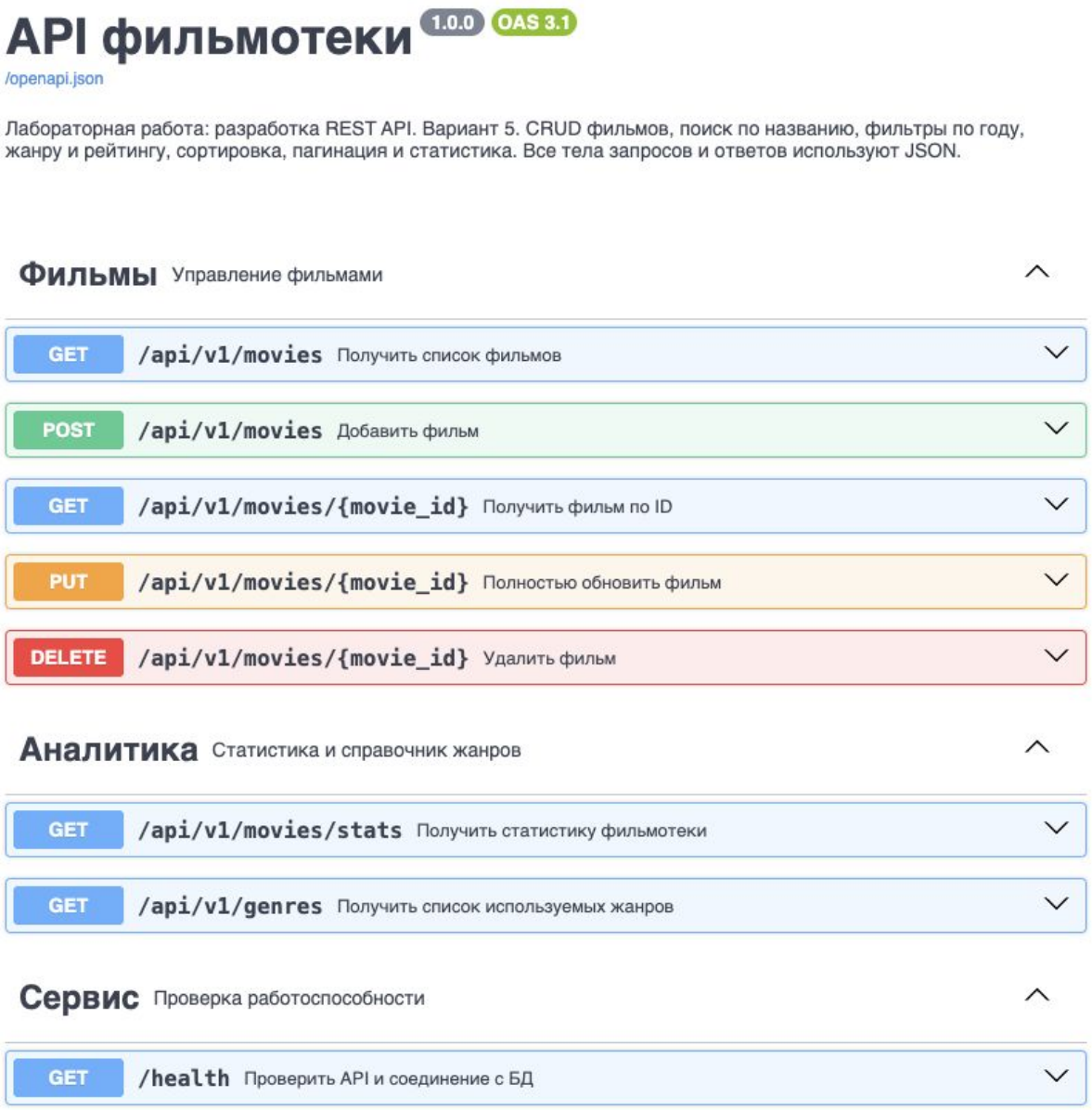


Рисунок 1 - фактический вид Swagger UI с восемью операциями API

9. Практическая проверка запросов

Примеры выполнены через TestClient на отдельной временной SQLite. Сначала загружены восемь учебных фильмов. Полные тела запросов и ответов сохранены в docs/demo-results.json; рабочая база при этом не изменяется.

Действие	HTTP-код	Наблюдаемый результат
GET /health	200	status=ok, database=sqlite
GET со списком и фильтрами	200	2 фильма, рейтинг по убыванию
POST контрольного фильма	201	Назначен id=9
GET созданного фильма	200	Возвращён созданный объект
PUT контрольного фильма	200	Рейтинг изменён на 9.0
DELETE контрольного фильма	204	Тело ответа отсутствует
GET удалённого фильма	404	Фильм не найден
POST с рейтингом 11	422	Ошибка поля body.rating

Пример тела POST-запроса, использованного при проверке:

```
{
  "title": "Контрольный фильм",
  "director": "Иван Петров",
  "year": 2024,
  "genre": "Фантастика",
  "rating": 8.4,
  "duration_minutes": 125,
  "description": "Учебный пример: экспедиция к далёкой планете."
}
```

Для исходного набора статистика вернула 8 фильмов, средний рейтинг 7.86, годы от 2020 до 2024. Агрегаты вычисляются в SQL с помощью COUNT, AVG, MIN, MAX и GROUP BY.

10. Автоматическое тестирование

Тестирование выполнено в Python 3.12.14. Версии основных пакетов: FastAPI 0.156.8, Pydantic 2.14.1, SQLAlchemy 2.0.55, Uvicorn 0.54.0, pytest 9.1.1. Прямые зависимости зафиксированы в requirements-файлах.

```
python -m pip install -r requirements-dev.txt
ruff check .
ruff format --check .
python -m pytest --cov=app --cov-report=term-missing
```

Группа проверок	Что проверено
CRUD	Создание, чтение, замена, удаление; Location и пустой ответ 204
Валидация	Типы, границы, пустые строки, лишние поля, неверные ID
Поиск и фильтры	Кириллица, регистр, AND, буквальные % и _, SQL-подобная строка
Сортировка и страницы	4 поля, оба направления, равные значения, отсутствие повторов
Агрегаты	Среднее, диапазон лет, группы жанров, пустая коллекция
Хранение	Чтение после перезапуска приложения и CHECK в SQLite
Документация	Доступность Swagger, ReDoc и 8 операций в OpenAPI
Демонстрационные данные	Повторное заполнение не создаёт копий

Результат: **66 тестов пройдено, ошибок и падений нет**. Покрытие исполняемых строк приложения без скрипта заполнения app.seed составило 99%: покрыто 187 из 188 строк. Проверки Ruff также завершились успешно.

Тесты используют независимые временные файлы SQLite. Проверка сохранения данных закрывает первое приложение и открывает второе с тем же файлом БД. GitHub Actions настроен на выполнение проверок при push и pull request.

Среда выводит одно предупреждение Starlette об устаревании HTTPX в TestClient. Оно не является ошибкой и не изменяет результаты. Фактический JUnit-отчёт находится в docs/test-results.xml.

11. Вывод

Разработан работающий REST API фильмотеки для варианта 5. Реализованы все четыре CRUD-операции, поиск по названию, фильтрация по году и рейтингу, дополнительный фильтр жанра и агрегирование данных. API содержит восемь операций, проверяет входные данные и предоставляет сортировку и пагинацию.

SQLite обеспечивает сохранение данных между запусками. ORM-модель отделена от схем ввода и вывода. Схемы Pydantic задают правила проверки и сериализации, а Swagger UI и ReDoc позволяют изучать контракт и выполнять запросы. Автоматические проверки подтвердили заявленное поведение.

В ходе работы применены принципы маршрутизации, HTTP-методы и коды, параметризованные запросы к БД, обработка ошибок и документирование OpenAPI. Цель лабораторной работы достигнута.

12. Комплект сдачи

В комплект входят исходный код Python, requirements-файлы, инструкция README, учебные данные, примеры HTTP-запросов, тесты, сохранённая спецификация OpenAPI, журнал демонстрационных ответов и настоящий PDF-отчёт.

Репозиторий для размещения: github.com/Ersan1908/prog_e.

Учебный проект рассчитан на локальное использование. Авторизация не добавлялась: соответствующее дополнительное требование выполнено сортировкой и пагинацией. При изменении схемы в дальнейших версиях потребуется добавить миграции базы.

13. Использованные источники

1. FastAPI. SQL (Relational) Databases.
https://fastapi.tiangolo.com/tutorial/sql-databases/

2. FastAPI. Testing.
https://fastapi.tiangolo.com/tutorial/testing/

3. SQLAlchemy 2.0. SQLite.
https://docs.sqlalchemy.org/en/20/dialects/sqlite.html

4. Pydantic. Fields.
https://docs.pydantic.dev/latest/concepts/fields/

Дата обращения к электронным источникам: 21.10.2026.